

Programação Aplicada em .Net

Aula 1

Prof. Me. Lucas Teodoro dos Santos

UNIP

10 de Agosto de 2026

Roteiro da aula

- 1 Apresentação da Disciplina
- 2 O que é Pattern Matching?
- 3 Tipos de Pattern Matching
- 4 Type Pattern
- 5 Pattern Matching com switch
- 6 Switch Expression
- 7 Constant Pattern
- 8 Relational Pattern
- 9 Logical Pattern
- 10 Guarda when
- 11 Property Pattern
- 12 Positional Pattern
- 13 Exercício de fixação

Nossa dinâmica em sala

Para que a disciplina seja produtiva, a aprendizagem precisa ser construída em conjunto.

- A participação durante as aulas é fundamental;
- O processo de aprendizagem depende de aproximadamente **50% do professor e 50% do aluno**;
- Não se aprende programação apenas assistindo outras pessoas programarem;
- Preparem seus computadores para acompanhar as atividades práticas durante a aula;
- Estarei disponível para esclarecer dúvidas e auxiliar durante o desenvolvimento das atividades.

Combinado da turma

Perguntem, participem, pratiquem e não deixem as dúvidas se acumularem.

Programação Aplicada em .NET

A disciplina aborda recursos modernos da linguagem C# e da plataforma .NET, com foco em organização de código, concorrência, desempenho, diagnóstico e boas práticas para o desenvolvimento de aplicações.

Conteúdo programático da disciplina:

- Linguagem avançada em C# e .NET;
- Performance e alocação eficiente;
- Concorrência e pipelines;
- Diagnóstico e perfilamento;
- Arquitetura aplicada;
- Tempo real e experiência do usuário;
- Qualidade contínua.

Avaliações da Disciplina

A nota de cada avaliação será composta por duas partes:

- **Trabalho:** Peso de **20%** da nota;
- **Prova:** Peso de **80%** da nota.

Composição da nota

$$\text{Nota} = (\text{Trabalho} \times 0,2) + (\text{Prova} \times 0,8)$$

Avaliação	Data
NP1	21/09/2026
NP2	09/11/2026

O que é Pattern Matching?

Pattern Matching é um recurso do C# que permite **verificar o tipo, o valor ou a estrutura de um dado** e executar uma ação conforme o padrão identificado.

Esse recurso:

- Torna as estruturas de decisão mais claras e expressivas;
- Reduz a necessidade de conversões explícitas (*casts*);
- Pode ser utilizado com `if`, `switch` e expressões `switch`;
- Permite combinar tipos, valores, propriedades e condições;
- Facilita o tratamento de diferentes formatos de dados.

Ideia principal

O Pattern Matching não verifica apenas se dois valores são iguais. Ele permite identificar se um dado possui determinado **tipo, valor, formato ou conjunto de características**.

Tipos de Pattern Matching

O C# oferece diferentes tipos de padrões para analisar valores, tipos e estruturas de objetos.

Entre os principais recursos estão:

- **Type Pattern:** verifica se um objeto corresponde a determinado tipo;
- **Pattern Matching com switch:** seleciona um bloco de código com base no padrão correspondente;
- **switch Expression:** avalia padrões e retorna diretamente um valor;
- **Constant Pattern:** verifica se um valor é igual a uma constante específica;
- **Relational Pattern:** compara um valor com uma constante utilizando operadores relacionais;
- **Logical Pattern:** combina padrões utilizando and, or e not;
- **Guarda when:** acrescenta uma condição adicional a um padrão;
- **Property Pattern:** verifica os valores das propriedades de um objeto;
- **Positional Pattern:** verifica os componentes de um objeto de acordo com suas posições.

Esses recursos podem ser combinados para construir decisões mais claras, concisas e expressivas.

Type Pattern

O **Type Pattern** verifica se um valor possui determinado tipo. Quando existe correspondência, o valor pode ser armazenado em uma variável já convertida para esse tipo.

```
1  object valor = 10;
2
3  if (valor is int numero)
4  {
5      Console.WriteLine($"Inteiro: {numero}");
6  }
```

Vantagem

A verificação do tipo e a conversão do valor são realizadas em uma única expressão, sem a necessidade de um *cast* explícito.

Type Pattern - Exemplo

Um sistema possui funcionários de diferentes tipos.

Crie as seguintes classes:

- Funcionario: classe base com as propriedades Nome e Idade;
- Desenvolvedor: herda de Funcionario e possui as propriedades Linguagem e AnosExperiencia;
- Gerente: herda de Funcionario e possui a propriedade TamanhoEquipe;
- Estagiario: herda de Funcionario e possui a propriedade HorasSemanais.

Crie um método que receba um object e informe se ele é um desenvolvedor, utilizando **Type Pattern**.

Solução com Type Pattern

```
1 static void VerificarFuncionario(object obj)
2 {
3     if (obj is Desenvolvedor desenvolvedor)
4     {
5         Console.WriteLine(
6             $"{desenvolvedor.Nome} e desenvolvedor.");
7
8         Console.WriteLine(
9             $"Linguagem: {desenvolvedor.Linguagem}");
10
11        Console.WriteLine(
12            $"Experiencia: {desenvolvedor.AnosExperiencia} anos");
13    }
14    else
15    {
16        Console.WriteLine(
17            "O objeto nao e um desenvolvedor.");
18    }
19 }
```

Pattern Matching com switch

O Pattern Matching pode ser utilizado com a estrutura `switch` para verificar diferentes padrões em um mesmo valor.

Cada `case` pode verificar o tipo do objeto e criar uma variável já convertida quando houver correspondência.

```
1  switch (objeto)
2  {
3      case Desenvolvedor desenvolvedor:
4          Console.WriteLine(desenvolvedor.Linguagem);
5          break;
6
7      case Gerente gerente:
8          Console.WriteLine(gerente.TamanhoEquipe);
9          break;
10 }
```

Permite organizar a verificação de vários tipos sem utilizar uma sequência extensa de `if` e `else if`.

Pattern Matching com `switch` - Exemplo

Crie um método que receba um parâmetro do tipo `object` e identifique se o objeto recebido é:

- Um Desenvolvedor;
- Um Gerente;
- Um Estagiário.

Para cada tipo de funcionário, apresente seu nome e uma informação específica:

- A linguagem do desenvolvedor;
- O tamanho da equipe do gerente;
- A quantidade de horas semanais do estagiário.

Utilize Pattern Matching com `switch` para realizar as verificações.

Pattern Matching com switch - Solução (1/2)

```
1 static void ExibirFuncionario(object obj)
2 {
3     switch (obj)
4     {
5         case Desenvolvedor desenvolvedor:
6             Console.WriteLine($"Nome: {desenvolvedor.Nome}");
7             Console.WriteLine($"Linguagem: {desenvolvedor.Linguagem}");
8             break;
9
10        case Gerente gerente:
11            Console.WriteLine($"Nome: {gerente.Nome}");
12            Console.WriteLine($"Tamanho da equipe:{gerente.TamanhoEquipe}");
13            break;
```

Pattern Matching com switch - Solução (2/2)

```
1      case Estagiario estagiario:
2          Console.WriteLine($"Nome: {estagiario.Nome}");
3          Console.WriteLine($"Horas semanais:{estagiario.HorasSemanais}");
4          break;
5
6      default:
7          Console.WriteLine("Objeto nao reconhecido.");
8          break;
9  }
10 }
```

Funcionamento

Cada case verifica o tipo do objeto e cria uma variável já convertida para acessar suas propriedades específicas.

switch Expression

A switch expression analisa um valor e retorna diretamente um resultado para o padrão correspondente.

Diferentemente do switch tradicional, ela não utiliza case e break. Cada alternativa utiliza o operador `=>`.

```
1  string resultado = objeto switch
2  {
3      Desenvolvedor dev => $"Desenvolvedor: {dev.Nome}",
4      Gerente gerente => $"Gerente: {gerente.Nome}",
5      Estagiario est => $"Estagiario: {est.Nome}",
6      _ => "Objeto nao reconhecido"
7  };
```

switch Expression - Exemplo

Crie um método que receba um parâmetro do tipo `object` e retorne uma descrição de acordo com o tipo de funcionário:

- Para `Desenvolvedor`, retorne seu nome e sua linguagem;
- Para `Gerente`, retorne seu nome e o tamanho da equipe;
- Para `Estagiario`, retorne seu nome e suas horas semanais;
- Para outros objetos, retorne uma mensagem informando que o objeto não foi reconhecido.

Utilize uma `switch expression` para produzir e retornar a descrição.

switch Expression - Solução

```
1 static string DescreverFuncionario(object obj)
2 {
3     string resultado = obj switch
4     {
5         Desenvolvedor dev =>
6             $"{dev.Nome}: linguagem {dev.Linguagem}",
7
8         Gerente gerente =>
9             $"{gerente.Nome}: equipe com {gerente.TamanhoEquipe} pessoas",
10
11         Estagiario estagiario =>
12             $"{estagiario.Nome}: {estagiario.HorasSemanais} horas semanais",
13
14         _ => "Objeto nao reconhecido"
15     };
16
17     return resultado;
18 }
```

Constant Pattern

O **Constant Pattern** verifica se um valor é igual a uma constante específica, como um número, um caractere, um texto ou null.

```
1 static string ClassificarStatus(int codigo)
2 {
3     return codigo switch
4     {
5         0 => "Operacao concluida",
6         1 => "Operacao pendente",
7         2 => "Operacao cancelada",
8         404 => "Registro nao encontrado",
9         _ => "Status desconhecido"
10    };
11 }
```

Comparação

Cada constante é comparada diretamente com o valor analisado. O padrão _ representa qualquer valor não tratado anteriormente.

Relational Pattern

O **Relational Pattern** permite comparar um valor com uma constante utilizando operadores relacionais.

Os operadores disponíveis são:

- < e <=;
- > e >=.

```
1  static string ClassificarExperiencia(int anos)
2  {
3      string resultado = anos switch
4      {
5          < 2 => "Iniciante",
6          >= 5 => "Experiente",
7          _ => "Intermediario"
8      };
9
10     return resultado;
11 }
```

É utilizado principalmente para classificar valores numéricos de acordo com limites ou faixas.

Classifique um desenvolvedor de acordo com seus anos de experiência:

- Menos de dois anos: desenvolvedor iniciante;
- De dois a quatro anos: desenvolvedor intermediário;
- Cinco anos ou mais: desenvolvedor experiente.

Crie um método que receba a quantidade de anos de experiência e retorne a classificação utilizando **Relational Pattern**.

Relational Pattern - Solução

```
1 static string ClassificarExperiencia(int anosExperiencia)
2 {
3     return anosExperiencia switch
4     {
5         < 2 => "Desenvolvedor iniciante",
6         < 5 => "Desenvolvedor intermediario",
7         >= 5 => "Desenvolvedor experiente"
8     };
9 }
```

Logical Pattern

O **Logical Pattern** permite combinar ou modificar padrões por meio dos operadores `and`, `or` e `not`.

- `and`: Exige que os dois padrões sejam atendidos;
- `or`: Exige que pelo menos um dos padrões seja atendido;
- `not`: Verifica se o padrão não é atendido.

```
1 static string ClassificarExperiencia(int anos)
2 {
3     return anos switch
4     {
5         >= 2 and < 5 => "Intermediario",
6         < 2 or > 10 => "Fora da faixa comum",
7         not 5 => "Experiencia diferente de 5",
8         _ => "Experiencia igual a 5"
9     };
10 }
```

Classifique um desenvolvedor de acordo com seus anos de experiência:

- Entre zero e um ano: desenvolvedor iniciante;
- Entre dois e quatro anos: desenvolvedor intermediário;
- Entre cinco e dez anos: desenvolvedor experiente;
- Menos de zero ou mais de dez anos: valor fora da faixa esperada.

Crie um método que receba a quantidade de anos de experiência e retorne a classificação utilizando **Logical Pattern**.

Logical Pattern - Solução

```
1 static string ClassificarExperiencia(int anosExperiencia)
2 {
3     return anosExperiencia switch
4     {
5         < 0 or > 10 => "Valor fora da faixa esperada",
6         >= 0 and < 2 => "Desenvolvedor iniciante",
7         >= 2 and < 5 => "Desenvolvedor intermediario",
8         >= 5 and <= 10 => "Desenvolvedor experiente"
9     };
10 }
```

Combinação dos padrões

O operador `and` define os limites de uma faixa, enquanto o operador `or` permite verificar condições alternativas.

Logical Pattern com not

O operador not pode ser combinado com is para verificar se um objeto não corresponde a determinado tipo.

```
1 static void VerificarFuncionario(object obj)
2 {
3     if (obj is not Desenvolvedor)
4     {
5         Console.WriteLine("Nao e um desenvolvedor.");
6         return;
7     }
8
9     Console.WriteLine("E um desenvolvedor.");
10 }
```

Nesse exemplo, a expressão `is not Desenvolvedor` será verdadeira quando o objeto não for do tipo `Desenvolvedor`.

Guarda when

A guarda when permite adicionar uma condição complementar a um padrão definido em um case.

O bloco será executado somente quando o padrão for atendido e a condição após o when for verdadeira.

```
1  switch (objeto)
2  {
3      case Desenvolvedor dev when dev.AnosExperiencia >= 5:
4          Console.WriteLine("Desenvolvedor experiente");
5          break;
6
7      case Desenvolvedor dev:
8          Console.WriteLine("Desenvolvedor");
9          break;
10 }
```

A guarda when permite diferenciar objetos que correspondem ao mesmo padrão utilizando condições adicionais.

Crie um método que receba um objeto do tipo `object` e retorne a classificação do funcionário de acordo com as seguintes condições:

- Desenvolvedor entre zero e dois anos de experiência: iniciante;
- Desenvolvedor entre três e quatro anos de experiência: intermediário;
- Desenvolvedor com cinco anos ou mais de experiência: experiente;
- Gerente com mais de dez funcionários: gerente de equipe grande;
- Nos demais casos: funcionário não classificado.

Utilize uma `switch expression`, a guarda `when` e padrões relacionais e lógicos.

Guarda when - Solução (1/2)

```
1 static string ClassificarFuncionario(object obj)
2 {
3     return obj switch
4     {
5         Desenvolvedor dev
6             when dev.AnosExperiencia is >= 0 and < 3
7                 => $"{dev.Nome}: desenvolvedor iniciante",
8
9         Desenvolvedor dev
10            when dev.AnosExperiencia is >= 3 and < 5
11                => $"{dev.Nome}: desenvolvedor intermediario",
12
13        Desenvolvedor dev
14            when dev.AnosExperiencia is >= 5
15                => $"{dev.Nome}: desenvolvedor experiente",
```

Guarda when - Solução (2/2)

```
1      Gerente gerente
2      when gerente.TamanhoEquipe > 10
3      => $"{{gerente.Nome}}: gerente de equipe grande",
4
5      _ => "Funcionario nao classificado"
6  };
7  }
```

Recursos utilizados

A guarda `when` adiciona condições aos padrões de tipo, enquanto `and` combina os padrões relacionais que definem os intervalos.

O **Property Pattern** permite verificar um objeto com base nos valores de suas propriedades.

As propriedades analisadas são indicadas entre chaves, sem a necessidade de acessá-las separadamente em uma guarda `when`.

Sintaxe

O nome da propriedade aparece antes de `:`, enquanto o padrão aplicado ao seu valor aparece depois.

Property Pattern - Exemplo

```
1  static string ClassificarFuncionario(object obj)
2  {
3      return obj switch
4      {
5          Desenvolvedor { AnosExperiencia: >= 5 }
6              => "Desenvolvedor experiente",
7
8          Gerente { TamanhoEquipe: > 10 }
9              => "Gerente de equipe grande",
10
11         _ => "Funcionario nao classificado"
12     };
13 }
```

No exemplo, o tipo do funcionário e o valor de uma de suas propriedades são verificados no mesmo padrão.

Crie um método que receba um funcionário e retorne sua classificação:

- Desenvolvedor de C# com cinco anos ou mais de experiência: especialista em C#;
- Gerente com mais de dez funcionários: gerente de equipe grande;
- Estagiário com até vinte horas semanais: estagiário de meio período;
- Nos demais casos: funcionário não classificado.

Utilize **Property Pattern** em uma `switch expression` para verificar o tipo do funcionário e os valores de suas propriedades.

Property Pattern - Solução

```
1 static string ClassificarFuncionario(object obj)
2 {
3     return obj switch
4     {
5         Desenvolvedor
6         {
7             Linguagem: "C#",
8             AnosExperiencia: >= 5
9         } dev => $"{dev.Nome}: especialista em C#",
10
11         Gerente { TamanhoEquipe: > 10 } gerente
12         => $"{gerente.Nome}: gerente de equipe grande",
13
14         Estagiario { HorasSemanais: <= 20 } estagiario
15         => $"{estagiario.Nome}: estagiario de meio periodo",
16
17         _ => "Funcionario nao classificado"
18     };
19 }
```

Positional Pattern

O **Positional Pattern** permite analisar os elementos de uma tupla de acordo com suas posições.

Em cada posição, é possível verificar um valor, identificar um tipo e capturar o elemento em uma variável.

```
1 static string Processar(object operacao, object valor)
2 {
3     return (operacao, valor) switch
4     {
5         ("dobrar", int numero)
6         => $"Resultado: {numero * 2}",
7
8         ("maiusculo", string texto)
9         => texto.ToUpper(),
10
11         _ => "Operacao nao reconhecida"
12     };
13 }
```

Positional Pattern - Exemplo

Crie um método que receba uma operação e um valor, ambos do tipo `object`, e processe as seguintes combinações:

- Operação “dobrar” e um número inteiro: retornar o dobro;
- Operação “maiusculo” e um texto: retornar o texto em letras maiúsculas;
- Operação “tamanho” e um texto: retornar a quantidade de caracteres;
- Uma operação conhecida com um tipo incompatível: informar a incompatibilidade;
- Qualquer outra combinação: dados não reconhecidos.

Utilize uma tupla, uma `switch expression` e Positional Pattern.

Positional Pattern - Solução

```
1 static string Processar(object operacao, object valor)
2 {
3     return (operacao, valor) switch
4     {
5         ("dobrar", int numero)
6         => $"Resultado: {numero * 2}",
7
8         ("maiusculo", string texto)
9         => texto.ToUpper(),
10
11        ("tamanho", string texto)
12        => $"O texto possui {texto.Length} caracteres",
13
14        (string comando, _)
15        => $"Operacao '{comando}' incompativel",
16
17        _ => "Dados nao reconhecidos"
18    };
19 }
```

Positional Pattern - Utilização

```
1 Console.WriteLine(Processar("dobrar", 10));  
2 // Resultado: 20  
3  
4 Console.WriteLine(Processar("maiusculo", "programacao"));  
5 // PROGRAMACAO  
6  
7 Console.WriteLine(Processar("tamanho", "CSharp"));  
8 // O texto possui 6 caracteres  
9  
10 Console.WriteLine(Processar("dobrar", "dez"));  
11 // Operacao 'dobrar' incompativel
```

Exercício de fixação

Desenvolva um sistema para classificar e processar pedidos utilizando Pattern Matching.

Crie uma classe base Pedido com as propriedades:

- Numero;
- Valor;
- QuantidadeItens.

Crie também as seguintes classes derivadas:

- PedidoOnline, com a propriedade Frete;
- PedidoLoja, com a propriedade RetiradaImediata;
- PedidoInternacional, com a propriedade PaisDestino.

Exercício de fixação – Classificação (1/2)

Crie o método:

```
ClassificarPedido(Pedido pedido)
```

O método deverá retornar:

- Pedido nulo: “Pedido inválido”;
- Pedido on-line com valor entre R\$ 0 e menor que R\$ 100: “Pedido online de baixo valor”;
- Pedido on-line com valor entre R\$ 100 e menor que R\$ 500: “Pedido online comum”;
- Pedido on-line com valor igual ou superior a R\$ 500 e frete gratuito: “Pedido online premium”.

Exercício de fixação – Classificação (2/2)

- Pedido de loja com retirada imediata: “Pedido disponível para retirada”;
- Pedido internacional destinado ao Brasil: “Pedido internacional para o Brasil”;
- Pedido internacional destinado a outro país: “Pedido internacional”;
- Qualquer outro caso: “Pedido não classificado”.

Utilize uma `switch expression`, `Property Pattern` e padrões relacionais e lógicos.

Exercício de fixação – Operações

Crie o método:

```
ProcessarOperacao(string operacao, Pedido pedido)
```

Analise a tupla (operacao, pedido) e retorne:

- enviar com PedidoOnline: informar o número;
- retirar com PedidoLoja: informar que o pedido está pronto;
- exportar com PedidoInternacional: informar o país de destino;
- Operação conhecida com tipo incompatível: informar a incompatibilidade;
- Qualquer outra operação: “Operação não reconhecida”.

Utilize Positional, Constant e Type Pattern.

Exercício de fixação – Quantidade

Crie o método:

```
ValidarQuantidade(int quantidade)
```

Classifique a quantidade de itens:

- Valor negativo: “Quantidade inválida”;
- Zero: “Pedido vazio”;
- Entre 1 e 5: “Poucos itens”;
- Entre 6 e 20: “Quantidade normal”;
- Acima de 20: “Muitos itens”.

Utilize padrões constantes, relacionais e lógicos.

Exercício de fixação – Requisitos

O programa deverá utilizar:

- Constant Pattern;
- Type Pattern;
- Relational Pattern;
- Logical Pattern com and, or ou not;
- Property Pattern;
- Positional Pattern com tupla;
- Guarda when;
- Padrões null e _;
- switch expression.

Crie pelo menos um objeto de cada tipo de pedido e teste todos os métodos, apresentando os resultados no console.

Obrigado!

Dúvidas?